Caleb Thow - Project Portfolio Page

Overview

Business Management System (BMS) is a Java command-line application for small, growing businesses.

It keeps product catalogues, stock, customer orders, sales and operating expenses in one place.

My contributions focused on expense and financial management, payment accuracy, command usability,

and the documentation needed to explain these features to users and future developers.

Summary of Contributions

- **Code contributed**: tP Code Dashboard

- **Feature: Expense management** (`expense add`, `expense list`)

  - What it does: records operating expenses against a product and lists recorded expenses in insertion order.

  - Justification: gives a business owner a simple way to record and inspect operating costs alongside sales.

  - Highlights: introduced the `Expense` and `ExpenseList` domain classes, parser support and dedicated commands; validates non-blank product names, non-negative amounts and at most two decimal places; preserves insertion order while returning a copy of the stored list so presentation code cannot mutate application state. Improved the confirmation and empty-list feedback so the CLI remains understandable for both successful and no-data cases (initial implementation and refinements in #76 and #77).

- **Feature: Financial reporting** (`finances`)

  - What it does: reports total order value, payments received, total expenses and profit or loss.

  - Justification: provides a compact view of the business's financial position without requiring manual calculation.

  - Highlights: separates active order value from payments received, excludes cancelled orders and assumed refunds, includes partial payments only to the amount received, and computes profit or loss as payments received minus expenses. Refactored monetary values to use `BigDecimal` throughout the relevant domain and UI paths, avoiding floating-point rounding errors in currency calculations. Added focused tests for profit, loss, equal totals and partial payments (#76).

- **Enhancement: Payment and currency correctness**

  - Updated customer-payment handling so payments are applied across eligible unpaid orders in history order, support partial settlement, and return excess as change. Kept the User Guide, order/payment tests and financial report consistent with the resulting rules.

  - Replaced `double`-based currency handling with `BigDecimal` in the expense and financial-reporting paths, with validation at the domain boundary and consistent two-decimal display. This was a cross-cutting change because order, product, stock, payment, UI and test code share monetary values.

- **Enhancement: Command usability**

  - Renamed the placeholder `bye` command to the clearer `exit` command and updated parsing, help text, tests and the User Guide (#78).

  - Aligned command ordering and the command summary so that the help output and User Guide present the same feature groups and syntax.

- **Testing and diagnostics**:

  - Added unit tests for `Expense`, `ExpenseList`, expense commands, parser behaviour and financial reports. Added Java assertions around expense-list operations and enabled assertions in Gradle test and run tasks (#43).

  - The tests cover validation boundaries, insertion order, display behaviour, partial payments, profit, loss, and the zero-difference case, reducing the risk of silently incorrect financial output.

- **Contributions to the User Guide**:

  - Wrote and refined the sections for adding/listing expenses and viewing the financial report, including syntax, validation rules, partial-payment semantics, examples and currency-formatting behaviour.

  - Updated the command summary, help/exit documentation, and related command descriptions as the CLI evolved.

- **Contributions to the Developer Guide**:
    - Wrote and reorganised the Expense and financial management section, documenting the command flow, class responsibilities, financial calculations, `BigDecimal` design choice, data-protection boundary and testing.
    - Designed and refined the expense/finance UML object and sequence diagrams for adding an expense, listing expenses, and generating the financial report. Corrected activation-bar placement, message alignment and UML notation across several iterations, then converted the final diagrams into portable SVG assets. Added contents and attribution structure to the guide (#79, #80, #81).
- **Contributions to team-based tasks**:
    - Helped establish and organise the team's GitHub organisation and repository workflow by enabling issue tracking and configuring labels so that work could be categorised and monitored consistently.
    - Maintained the issue tracker, helped the team track deadlines and coordinate outstanding tasks, and supported release planning so that deliverables stayed on schedule through to completion.
    - Updated team-wide project documentation, removed obsolete placeholder command documentation, and fixed documentation/build compatibility issues when features were integrated. Contributed general implementation and documentation changes across the project, and helped review and close pull requests.
    - Coordinated cross-feature changes so payment, order, product, stock, help and finance code continued to use consistent money types and command names.
- **Review/mentoring contributions**:
    - Reviewed the team's financial-reporting and expense changes, including checks for payment semantics, assertions and help-text consistency. Relevant integration work is recorded in #76, #77 and #78.
- **Contributions beyond the project team**:
    - Performed exploratory testing, raised issues for bugs discovered during development, and helped verify that fixes addressed the reported behaviour.
    - Reviewed another group's project, focusing mainly on the Developer Guide, and provided comments on its organisation, clarity and technical documentation in PR #6.